

3. Algoritmi di ordinamento

Algoritmi di ordinamento

- Ordinamento interno → dati da ordinare in memoria centrale → accesso diretto agli elementi (array)
- Ordinamento esterno → dati da ordinare in memoria di massa → accesso a blocchi di dati (latenza HW periferiche)

tecniche differenti

Cosa ordiniamo?

- vettori di strutture complesse come oggetti o record
- un particolare campo è scelto come *chiave* per l'ordinamento

Stabilità

 Un algoritmo di ordinamento è detto *stabile* se preserva l'ordine tra record con la medesima chiave

È fondamentale quando si fanno ordinamenti multipli.
Esempio: prima ordini per cognome, poi per nome. Se il secondo ordinamento è stabile, le persone con lo stesso nome restano ordinate per cognome.

Complessità degli algoritmi di ordinamento

- **spazio** si considera la memoria utilizzata in aggiunta all'array da ordinare (anche lo stack in caso di algoritmi ricorsivi)
- **tempo** stima della complessità di tempo in funzione della lunghezza del vettore da ordinare, calcolando come prima cosa il *numero di confronti tra chiavi*.

→ algoritmi di ordinamento basati su confronti tra chiavi.

Il numero di confronti è l'operazione più costosa utilizzata da questi algoritmi.

- Se ciascun confronto avviene in tempo costante, il numero di confronti fornisce una stima del tempo di calcolo.
- *In generale:* il tempo di calcolo può essere stimato moltiplicando il numero di confronti per il tempo di utilizzo da ciascun confronto.

Techine di ordinamento interno

Algoritmi elementari

`#n_cfr` = $\Theta(n^2)$

- per selezione - `selectionSort`
- per inserimento - `insertionSort`
- "a bolle" - `bubbleSort`

Algoritmi avanzati

`#n_cfr` = $\Theta(n \log n)$

- per fusione - `mergeSort`
- veloce - `quickSort` - *worst case* $\Theta(n^2)$
- basato su "heap" - `heapSort`

Sono algoritmi *in loco* - non necessitano di strutture ausiliarie, eccetto `mergeSort, quickSort`

Non tutti sono stabili.

Selection Sort

l'algoritmo *ad alto livello* funziona in questo modo:

1. scorrere l'intero vettore per trovare l'elemento più piccolo (questo richiede $n - 1$ confronti)
2. lo sposta nella prima posizione
3. scorre il resto del vettore ($n - 1$ elementi) per trovare il più piccolo (richiede $n - 2$ confronti)
4. lo sposta nella seconda posizione
5. continua così fino alla fine

```
ALGORITMO selectionSort(array A[0...n - 1])
  FOR k ← 0 TO n - 2 DO
    // ricerca la posizione del minimo in A[k...n - 1]
    m ← k
    FOR j ← k + 1 TO n - 1 DO
      IF A[j] < A[m] THEN
        m ← j
    scambio A[m] con A[k]
```

Tempo

Numero di confronti:

$$\sum_{k=0}^{n-2} (n - k - 1) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

→ $\Theta(n^2)$

Spazio

Costante → $O(1)$

algoritmo non stabile.

Insertion Sort



L'algoritmo funziona prendendo un elemento alla volta (la "chiave") e inserendolo al posto giusto nella sotto-lista *già ordinata* alla sua sinistra. Per fare questo, confronta la chiave con gli elementi a sinistra, spostandoli a destra finché non trova la posizione corretta.

```
ALGORITMO insertionSort(array A[0...n-1])
  FOR k ← 1 TO n - 1 DO
    x ← A[k]
    j ← k - 1
    WHILE j ≥ 0 AND A[j] > x DO
      A[j + 1] ← A[j]
      j ← j - 1
    A[j + 1] ← x
```

Alla *k-esima* iterazione:

- al massimo *k* confronti
- al minimo 1

Tempo

$$\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} = \Theta(n^2)$$

- caso peggiore: array ordinato al contrario
- caso migliore: array già ordinato *n - 1* confronti

spazio

costante - $O(1)$

algoritmo stabile.



L'algoritmo è molto veloce su dati quasi ordinati - **comportamento adattivo**

Bubble Sort

Versione “non giusta”

```
ALGORITMO bubbleSort(array A[0...n - 1])
  DO
    scambiato ← false
    FOR j ← 1 TO n - 1 DO
      IF A[j] < A[j - 1] THEN
        scambia A[j] e A[j - 1]
        scambiato ← true
    WHILE scambiato
```

Posso sfruttare il fatto che l'ultimo elemento viene sempre posto in fondo in ogni iterazione

Versione “corretta”



L'algoritmo `bubbleSort` "ottimizzato" prevede una bandierina (flag) che controlla se in un intero ciclo interno è avvenuto almeno uno scambio. Se un intero ciclo si conclude senza scambi, significa che l'array è ordinato e l'algoritmo si ferma in anticipo.

```
ALGORITMO bubbleSort(array A[0...n - 1])
  i ← 1
  DO
    scambiato ← false
    FOR j ← 1 TO n - i DO
      IF A[j] < A[j - 1] THEN
        scambia A[j] e A[j - 1]
        scambiato ← true
    i ← i + 1
  WHILE scambiato AND i < n
```

Caso peggiore: array al contrario

$$\sum_{i=1}^{n-1} n - i = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} = \Theta(n^2) \text{ confronti}$$

Caso migliore: array già ordinato

n - 1 confronti

- spazio $O(1)$
- Algoritmo stabile
- se nel confronto ci fosse `<=` farei scambi inutili!
- **confronta coppie di dati adiacenti**

 Lezione 8 - @October 10, 2025

Quale algoritmo elementare fa meno spostamenti?

- selection sort - effettua spostamenti nell'ordine di n
- se scambio degli *oggetti* lo scambio potrebbe essere costoso, ma spesso nell'array ci sono solo puntatori
- devo capire *cosa sto scambiando*

Merge - fusione di un array

Dati due array ordinati, voglio ottenere un nuovo array ordinato con gli stessi elementi

- posso mettere nel nuovo array gli elementi del secondo array in coda agli elementi del primo e poi applico un algoritmo di ordinamento noto - **NO! non sto sfruttando l'ipotesi che i vettori siano già ordinati.**

Idea

- due puntatori - uno per array
- un altro puntatore per il nuovo array

Algoritmo

```
ALGORITMO merge(array B[0...lb - 1], array C[0...lc - 1]) → array
  Sia X[0...lb + lc - 1] un array
  i1 ← 0, i2 ← 0, k ← 0
  WHILE i1 < lb AND i2 < lc DO // in ognuno dei due array è rimasto qualcosa
    IF B[i1] ≤ C[i2] THEN
      X[k] ← B[i1]
      i1 ← i1 + 1
    ELSE
      X[k] ← C[i2]
      i2 ← i2 + 1
    k ← k + 1
  // non abbiamo finito - sono avanzati degli elementi in B o C
  IF i1 < lb THEN
    FOR j ← i1 to lb - 1 DO
      X[k] ← B[j]
      k ← k + 1
  ELSE // è avanzato qualcosa in C
    FOR j ← i2 to lc - 1 DO
      X[k] ← C[j]
      k ← k + 1
  RETURN X
```

Quanti confronti vengono fatti?

Se $l_B + l_C = n$

- un confronto mi permette di scegliere un elemento e metterlo in x
- quanti elemento scelgo? $n \rightarrow$ i confronti sono al massimo $n - 1$.
- nel caso peggiore avanza un elemento.

Tecnica *divide-et-impera*

P Problema

I istanza di P

- se l'istanza è piccola risolvi direttamente il problema altrimenti
 - dividere istanze di lunghezza separatamente
 - ricava la soluzione del problema combinando le varie soluzioni

```
ALGORITMO risolviP(istanza I) → soluzione
  IF |I| ≤ C THEN
    risolvi P su I direttamente
    RETURN soluzione
  ELSE
    dividi I in m istanze I1, I2, ..., Im con |Ij| ≤ |I| per j = 1 ... m
    sol1 ← risolviP(I1)
    sol2 ← risolviP(I2)
    ...
    solm ← risolviP(Im)
    RETURN combina(sol1, sol2, ..., solm)
```

Per valutare il tempo → equazione di ricorrenza

```
ALGORITMO mergeSort(array A[0...n - 1])
  IF n ≤ 1 THEN
    A è già ordinato
  ELSE
    - dividi A in due parti della stessa lunghezza
    - ordina le due parti separatamente
    - fondi le due parti ordinate (merge)
```

$$T(I) = \begin{cases} \text{costante} & |I| \leq C \\ T_{\text{dividi}}(I) + T(I_1) + T(I_2) + \dots + T(I_m) + T_{\text{combina}}(sol_1, sol_2, \dots sol_m) & \text{altrimenti} \end{cases}$$

Algoritmo Merge Sort

Idea

Pseudo-codice

```
ALGORITMO mergeSort(array A[0...n - 1])
  IF n > 1 THEN
    m ← n/2
    B ← [0...m - 1] // prima metà di A
    C ← [m...n - 1] // seconda metà di A
    mergeSort(B)
    mergeSort(C)
    A ← merge(B, C)
```

$C(n)$ = nr di confronti per ordinare un array di n elementi

$$C(n) = \begin{cases} 0 & n \leq 1 \\ C\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + C\left(\left\lceil \frac{n}{2} \right\rceil\right) + C_{\text{merge}}(n) & \text{altrimenti} \end{cases}$$

- $C\left(\left\lfloor \frac{n}{2} \right\rfloor\right)$ - mergeSort(B)
- $C\left(\left\lceil \frac{n}{2} \right\rceil\right)$ - mergeSort(C)

per n pari

$$C(n) = \begin{cases} 0 & n \leq 1 \\ 2 \cdot C\left(\frac{n}{2}\right) + n - 1 & \text{altrimenti} \end{cases}$$

Somma prime k potenze di 2

$$\sum_{i=0}^{k-1} 2^i = 2^k - 1$$

In binario

			1
		1	0
	1	0	0
1	0	0	0
..	..	..	..
1	1	1	1

tra ogni numero e il suo doppio esiste la potenza di due

$$\forall n \in \mathbb{N} \exists \text{potenza di } 2 \ N \mid n \leq N \leq 2n$$

Numero confronti mergeSort

$$C(n) = \begin{cases} 0 & n \leq 1 \\ C\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + C\left(\left\lceil \frac{n}{2} \right\rceil\right) + C_{\text{merge}}(n) & \text{altrimenti} \end{cases}$$

per n pari

$$C(n) = \begin{cases} 0 & n \leq 1 \\ 2 \cdot C\left(\frac{n}{2}\right) + n - 1 & \text{altrimenti} \end{cases}$$

Svolgendo l'equazione

$$C(n) = 2C\left(\frac{n}{2}\right) + n - 1 = 2\left[2C\left(\frac{n}{2^2}\right) + \frac{n}{2} - 1\right] = 2^2C\left(\frac{n}{2^2}\right) + n - 2 + n - 1 = 2^2\left[2C\left(\frac{n}{2^3}\right) + \frac{n}{2^2} - 1\right] + n - 2 + n - 1 = 2^3C\left(\frac{n}{2^3}\right) + n - 2^2 + n - 2 + n$$

se $\frac{n}{2^k} = 1 \rightarrow n = 2^k \rightarrow k = \log_2 n$

$$n \cdot C(1) + n \log_2 n - n + 1 = n \log_2 n - n + 1$$

$C(n) = n \log_2 n - n + 1 \rightarrow$ numero di confronti nel caso peggiore. Con ipotesi che n sia una potenza di 2.

Conclusioni

- per n potenza di 2
 - $C(n) = \Theta(n \log n)$
- in generale - $\exists N$ potenza di 2 | $n \leq N \leq 2n$

»

→ in ogni caso $C(n) = \Theta(n \log n)$

Analisi dello spazio

$H(n)$: altezza dello stack per la ricorsione

- $n = 1 \rightarrow H(1)$
- $n > 1 \rightarrow H(n) = 1 + \max \left\{ H \left(\left\lfloor \frac{n}{2} \right\rfloor \right), H \left(\left\lceil \frac{n}{2} \right\rceil \right) \right\}$

per n pari

$$H(n) = \begin{cases} 1 + H\left(\frac{n}{2}\right) & n > 1 \\ 1 & \text{altrimenti} \end{cases}$$

- per n potenza di 2:
 - $H(n) = 1 + \log_2 n$
- in generale
 - $H(n) = 2 + \log_2 n$

→ $H(n) = \Theta(\log n)$

Implementazione **mergeSort**

Se si implementa direttamente l'algoritmo nel modo in cui è stato scritto, ad ogni chiamata sull'array **A** di lunghezza > 1 :

- si creano due nuovi array **B**, **C**
→ spreco di spazio
- si copia in essi il contenuto di **A**
→ spreco di tempo

→ **soluzione alternativa**: con un unico array ausiliario **X** per il merge.

```
ALGORITMO mergeSort(array A[0...n - 1])
    Sia X[0...n - 1] un array
    mergeSort(A, 0, n, X)

PROCEDURA mergeSort(array A, indice i, indice f, array X)
    IF f - i > 1 THEN
        m ← (i + f)/2
        mergeSort(A, i, m, X)
        mergeSort(A, m, f, X)
        merge(A, i, m, X)

PROCEDURA merge(array A, indice i, indice m, indice f, array X)
    i1 ← i, i2 ← m, k ← 0
    WHILE i1 < m AND i2 < f DO // in entrambi le parti è restato qualcosa
        IF A[i1] < A[i2] THEN
            X[k] ← A[i1]
            i1 ← i1 + 1
        ELSE
            X[k] ← A[i2]
            i2 ← i2 + 1
        k ← k + 1
    FOR j ← i1 TO m - 1 DO // copia in X gli elementi rimasti della prima parte
        X[k] ← A[j]
        k ← k + 1
    FOR j ← i2 TO f - 1 DO // copia in X gli elementi rimasti della seconda parte
        X[k] ← A[j]
        k ← k + 1

    // copia il risultato del merge in A[1...f - 1]
    FOR k ← 0 TO f - i - 1 DO
        A[i + k] ← X[k]
```

Esempio: dato un array trovare l'elemento massimo e l'elemento minimo

algoritmo che riceve un array e restituisce l'elemento massimo e minimo.

#nr_cfr = $2n - 2$

→ Possiamo fare di meglio?

```
FUNZIONE minMax(array A, indice i, indice f) → (elemento, elemento)
IF f − i = 1 THEN RETURN (A[i], A[i])
ELSE IF f − i = 2 THEN
    IF A[i] < A[i + 1] THEN RETURN (A[i], A[i + 1])
    ELSE RETURN (A[i + 1], A[i])
ELSE
    m ← (i + f)/2
    (min1, max1) ← minMax(A, i, m)
    (min2, max2) ← minMax(A, m, f)
    IF min1 < min2 THEN min ← min1
    ELSE min ← min2
    IF max1 > max2 THEN max ← max1
    ELSE max ← max2

    RETURN (min, max)

ALGORITMO minMax(array A[0...n]) → (elemento, elemento)
RETURN minMax(A, 0, n)
```

$$C(n) \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ C\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + C\left(\left\lceil \frac{n}{2} \right\rceil\right) + 2 & \text{altrimenti} \end{cases}$$

Se **n** pari

$$2C\left(\frac{n}{2}\right) + 2$$

Per *n* potenza di 2

$$C(n) = 2C\left(\frac{n}{2}\right) + 2 = 2\left[2C\left(\frac{n}{2^2}\right) + 2\right] + 2 = 2^2C\left(\frac{n}{2^2}\right) + 2^2 + 2 = 2^2\left[2C\left(\frac{n}{2^3}\right) + 2\right] + 2^2 + 2 = 2^3C\left(\frac{n}{2^3}\right) + 2^3 + 2^2 + 2 = \dots =$$

$$2^kC\left(\frac{n}{2^k}\right) + \sum_{i=1}^k 2^i = 2^kC\left(\frac{n}{2^k}\right) + 2^{k+1} - 2$$

Se $\frac{n}{2^k} = 2 \rightarrow n = 2^{k+1} \rightarrow k = \log_2 n - 1$

$$= \frac{n}{2}C(2) + n - 2$$

$$C(n) = \frac{3}{2}n - 2$$

esercizio: per n non potenza di 2

- scrivo $n = 2^j + n'$ con $n' < 2^j$
- si ottiene $C(n) = \left\lceil \frac{3}{2}n \right\rceil - 2$

→ risparmio circa il 25%

Equazione di ricorrenza

$$F(n) = \begin{cases} b' & n = 1 \\ m \cdot F\left(\frac{n}{a}\right) + b''n^c & \text{altrimenti} \end{cases}$$

con $m, a, b', b'', c \in \mathbb{R}^+ \ a > 1$

$$F(n) = \begin{cases} \Theta(n^c) & m < a^c \\ \Theta(n^c \log n) & m = a^c \\ \Theta(n^{\log_a m}) & m > a^c \end{cases}$$

Il risultato può essere esteso anche nel caso in cui per $n > 1$ l'equazione data sia nella forma

$$F(n) = m_1F\left(\left\lfloor \frac{n}{a} \right\rfloor\right) + m_2F\left(\left\lceil \frac{n}{a} \right\rceil\right) + b''n^c$$

con $m = m_1 + m_2$

Esempio: Master Theorem

$$H(n) = \begin{cases} 1 & n = 1 \\ H\left(\frac{n}{2}\right) + 1 & \text{altrimenti} \end{cases}$$

- $m = 1$
- $a = 2$
- $c = 0$
- $a^c = 1$
- $H(n) = \Theta(\log n)$

Altri esempi sulle slides

Operazioni fra matrici

Somme matrici quadrate

n x n

Costo n^2

Prodotto matrici quadrate

c_ij = sum_{k=1}^n a_ik * b_kj

- n prodotti
- n - 1 somme

-> 2n - 1 operazioni elementari per ciascun elemento

#tot_op : n^2(2n - 1) = 2n^3 - n^2 = Theta(n^3)

upper bound: Theta(n^3)

lower bound: Omega(n^2) -> nessuno fino ad ora e riuscito a dimostrare che servano piu operazioni di cosi

Mentre esistono delle tecniche che non usano piu la definizione ma che riesce a ridurre un po' l'upper bound.

divide et impera

Lavoreremo con matrici quadrate n x n con n potenza di 2

1. scomposizione della matrice in sotto-matrici
 - a. se n = 1 -> C = [a_11 * b_11]
 - b. se n > 1 -> calcolo ricorsivamente

T(n) = #operazioni per moltiplicare

Algoritmo divide-et-impera immediato

Calcolo numero operazioni

T(n) = { 1 n = 1, 8T(n/2) + n^2 altrimenti }
T(n) = Theta(n^3)

Algoritmo di Strassen

Date A = [A11 A12; A21 A22] e B = [B11 B12; B21 B22]

Considero le matrici:

- M1 = (A21 + A22 - A11) * (B22 - B12 + B11)
- M2 = A11 * B11
- M3 = A12 * B21
- M4 = (A11 - A21) * (B22 - B11)
- M5 = (A21 + A22) * (B12 - B11)
- M6 = (A12 - A21 + A11 - A22) * B22
- M7 = A22 * (B11 + B22 - B12 - B21)

Si puo verificare che:

C = A * B = [M2 + M3 M1 + M2 + M5 + M6; M1 + M2 + M4 - M7 M1 + M2 + M4 + M5]

Handwritten notes: 7 prodotti di matrici n/2 x n/2, 24 somme/sottrazioni di matrici n/2 x n/2

- nel caso in cui n = 1 -> 1 operazione
- altrimenti
 - 7 prodotti di matrici n/2 x n/2 -> 7T(n/2)
 - 24 somme/sottrazioni di matrici n/2 x n/2 -> 24(n/2)^2 = 6n^2

Calcolo di T(n)

T(n) = { 1 n = 1, 7T(n/2) + 6n^2 altrimenti }

-> T(n) = Theta(n^{log_2 7}) = Theta(n^{2.8})

Quick Sort

algoritmo basato su divide-et-impera

$n =$ #elementi

- caso semplice $\rightarrow$ soluzione immediata $n \leq 1$
- altrimenti
 - dividi l'array in 2 parti
 - ordina le due parti separatamente
 - combina le soluzioni

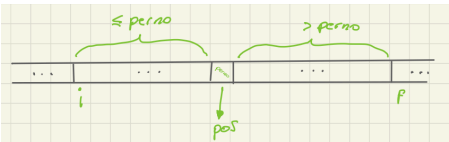
perno - pivot, scelto arbitrariamente

Pseudo-codice

```
ALGORITMO quickSort(array A)
  IF lunghezza di A > 1 THEN
    scegli un elemento x in A      //
    B <- {y in A | y < x}          // partizionamento
    C <- {y in A | y > x}          //
    quickSort(B)
    quickSort(C)
    A <- B, x; C
```

Come scelgo la modalità di partizionamento del vettore?

```
ALGORITMO partiziona(array A, indice i, indice f) -> indice
- partiziona A[i...f-1] rispetto a un elemento di A[i...f-1]
  scelto come perno, spostando gli elementi, in che alla fine
```



- restituisce la posizione finale del perno (pos)

Una volta scelto il pivot la sua posizione è definitiva

Implementazione in pseudo-codice

```
ALGORITMO partiziona(array A, indice i, indice f) → indice
  perno ← A[i]
  sx ← i
  dx ← f
  WHILE sx < dx DO
    DO dx ← dx - 1 WHILE A[dx] > perno
    DO sx ← sx + 1 WHILE sx < dx AND A[sx] ≤ perno
    IF sx < dx THEN
      scambia A[sx] con A[dx]
  scambia A[i] con A[dx]
  RETURN dx
```

Quick Sort pseudo codice finale

```
PROCEDURA quickSort(array A, indice i, indice f)
  IF f - i > 1 THEN
    m ← partiziona(A, i, f)
    quickSort(A, i, m)
    quickSort(A, m + 1, f)

ALGORITMO quickSort(array A[0...n - 1])
  quickSort(A, 0, n)
```

Quick sort - numero di confronti

- se $n \leq 1 \rightarrow 0$ confronti
- altrimenti
 - confronti per calcolare partizione $\rightarrow C_{\text{part}}(n) = n$
 - confronti per ordinare parte $sx \rightarrow C(k)$
 - confronti per ordinare $dx \rightarrow C(n - k - 1)$

Caso peggiore

equazione di ricorrenza

$$C_w(n) = \begin{cases} 0 & n \leq 1 \\ n + \max\{C_w(k) + C_w(n - k - 1) | k = 0 \dots n - 1\} & \text{altrimenti} \end{cases}$$
$$\begin{aligned} C_w(n) &= n + C_w(n - 1) \\ &= n + n - 1 + C_w(n - 2) = n + n - 1 + n - 2 + C_w(n - 3) \\ &= n + n - 1 + n - 2 + \dots + 2 + C_w(1) \end{aligned}$$

$$= \sum_{i=2}^n i = \frac{n(n+1)}{2} - 1 = \Theta(n^2)$$

Caso migliore

$$C_w(n) = \begin{cases} 0 & n \leq 1 \\ n + \min\{C_w(k) + C_w(n-k-1) | k = 0 \dots n-1\} & \text{altrimenti} \end{cases}$$

partizione migliore per $k \approx \frac{n}{2}$

$$C_b(n) = n + 2C_b\left(\frac{n}{2}\right)$$

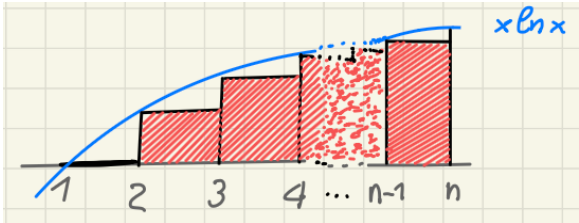
Dunque

$$C_b(n) \approx n \log_2 n$$

Problema

trovare una limitazione superiore per

$$\sum_{i=2}^{n-1} i \ln i$$



$$\sum_{i=2}^{n-1} i \ln i \leq \int_2^n x \ln x dx$$

integrazione per parti...

$$\sum_{i=2}^{n-1} i \ln i \leq \frac{n^2}{2} \ln n - \frac{n^4}{4}$$

Caso medio

$$C(n) = \begin{cases} 0 & n \leq 1 \\ \frac{1}{n} \sum_{k=0}^{n-1} [n + C(k) + C(n-k-1)] & \text{altrimenti} \end{cases}$$

Per $n \geq 2$

$$C(n) = \frac{1}{n} \sum_{k=0}^{n-1} n + \frac{1}{n} \sum_{k=0}^{n-1} C(k) + \frac{1}{n} \sum_{k=0}^{n-1} C(n-k-1)$$



$$\sum_{k=0}^{n-1} C(k) = \sum_{k=0}^{n-1} C(n-k-1)$$



$$n + \frac{n}{2} \sum_{i=0}^{n-1} C(i)$$

Equazione di ricorrenza trovata

$$C(n) = \begin{cases} 0 & n \leq 1 \\ n + \frac{n}{2} \sum_{i=2}^{n-1} C(i) & \text{altrimenti} \end{cases}$$

la sommatoria parte da $i = 2$ poiché per $C(0) = C(1) = 0$.

verifica che $C(n) \leq 2n \ln n$ per $n \geq 1$

Dimostrazione per induzione

- base $n = 1$

$$\begin{aligned} C(1) &= 0 \\ 2n \ln n &= 2 \ln 1 = 0 \\ \text{OK.} \end{aligned}$$

- induzione $C(n) = n + \frac{2}{n}$

$$\begin{aligned} C(n) &= n + \frac{2}{n} \sum_{i=2}^{n-1} C(i) \leq n + \frac{2}{n} \sum_{i=2}^{n-1} (2i \ln i) \\ &= n + \frac{2}{n} \cdot 2 \sum_{i=2}^{n-1} i \ln i \leq n + \frac{4}{n} \left(\frac{n^2}{2} \ln n - \frac{n^2}{4} \right) \\ &= n + 2n \ln n - n = 2n \ln n \approx 1.39n \log_2 n \end{aligned}$$

→ #cfr_in_media $C(n) \approx 1.39n \log_2 n$

Dalla lezione 11. → Quick Sort

	$C(n)$
caso peggiore	$\frac{n^2}{2}$
caso migliore	$n \log_2 n$
caso medio	$1.39n \log_2 n$

← molto raro

Come evitare il caso peggiore → scambiare in maniera random gli elementi tra di loro oppure con altre tecniche

Quanta memoria occupa l’algoritmo?

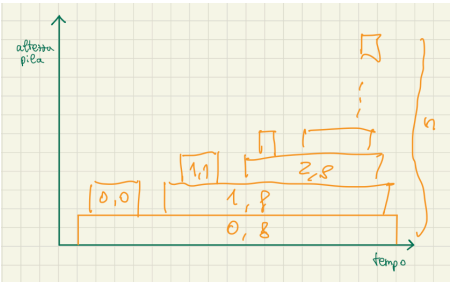
spesso viene definito *in loco* oppure no, se si considera lo stack per la gestione della ricorsione l’algoritmo è **in loco**.

partiziona

- numero costante di variabili

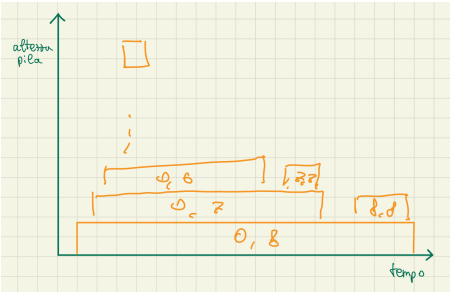
quickSort procedura ricorsiva

- variabile aggiuntiva ma lo stack cresce siccome ricorsiva.
- analisi casi peggiori - *quando ci mettiamo tanti confronti, anche lo stack usato è tanto!*
 - a. array già ordinato con primo elemento scelto come *pivot*
altezza stack $\approx n$



- b. array quasi ordinato

altezza stack $\approx n$



→ In generale quando si verifica un caso peggiore nel numero di confronti, l’andamento è analogo nello stack.

Si può evitare la ricorsione?

```
PROCEDURA quickSort(array A, indice i, indice f)
  WHILE f-i > 1 DO
    m <- partiziona(A, i, f)
    quickSort(A, i, m)
    i <- m + 1
```

→ evitiamo così la seconda chiamata ricorsiva.

Se ordino prima la parte dx invece che sx?

```
PROCEDURA quickSort(array A, indice i, indice f)
  WHILE i-f > 1 DO
    m <- partiziona(A, i, f)
    quickSort(A, m+1, f)
    quickSort(A, i, m)
```

→ scambia chiamate ricorsive

```
PROCEDURA quickSort(array A, indice i, indice f)
  WHILE f-i > 1 DO
    m <- partiziona(A, i, f)
    quickSort(A, m+1, f)
    f <- m
```

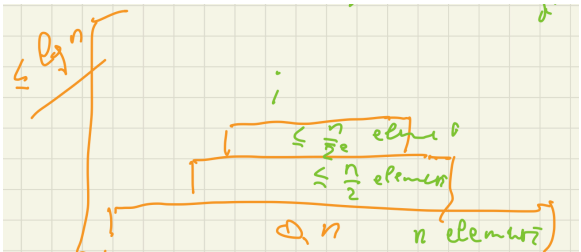
Ho molte versioni. → scelgo ogni volta quella più comoda

```
PROCEDURA quickSort(array A, indice i, indice f)
  WHILE f-i > 1 DO
    m <- partiziona(A, i, f)
    IF m-1 < f-m THEN // scelgo prima versione
```

```
quickSort(A, i, m)
i <- m + 1
ELSE // scelgo la seconda versione
quickSort(A, m+1, f)
f <- m
```

Chiamata ricorsiva sulla parte dell'array più piccola.

La parte ricorsiva è lunga $\leq \frac{n}{2}$... altezza $\leq \log_2 n$



Ricapitolando

nella versione "migliorata":

- altezza stack $\Theta(\log n)$
- grandezza record di attivazione $O(1)$
- $\rightarrow$ spazio totale $\Theta(\log n)$

versione base

- altezza stack $\Theta(n)$
- grandezza record di attivazione $O(1)$
- $\rightarrow$ spazio totale $O(n)$

 l'algoritmo **non** è **stabile** (poco rilevante)